

CAP theorem evaluation on Redis: Availability, Consistency, and Partitioning

MASTER DEGREE IN COMPUTER SCIENCE

Big Data Analytics – Academic Year 2025/2026

Author:

Pellacani Simone (ID: 217424)

19 giugno 2026

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



Sommario

Questo rapporto descrive una valutazione sperimentale di un sistema distribuito NOSQL basato su Redis. L'obiettivo è quantificare il comportamento del sistema sotto guasti controllati e partizioni di rete, e collegare le osservazioni al teorema CAP. Il codice dei test e gli script di orchestrazione sono disponibili nel repository del progetto (nota a piè di pagina della pagina del titolo) e su <https://github.com/USERNAME/REPOSITORY>.

Indice

Sommario	1
1 Requisiti	2
2 Obiettivi	2
3 Cosa testeremo: teorema CAP e modelli NoSQL	2
4 Redis data model e implicazioni per i test	2
4.1 Tipi principali e proprietà	2
4.2 Replicazione, persistenza e implicazioni di consistenza	2
5 Setup sperimentale e metodologia dei test	3
5.1 Test generico	3
6 Modelli di dati in Redis	3
6.1 Panoramica	3
6.2 Tipi principali e proprietà	3
6.3 Data Model example	3
6.4 Replicazione, persistenza e implicazioni di consistenza	4
7 Setup sperimentale e metodologia dei test	4
7.1 API e workload	4
7.1.1 Dettagli dei test	5
8 Discussione	5
9 Conclusioni	5
Riferimenti	6

1 Requisiti

- **Funzionali:** deploy Redis multinodo capace di accettare letture e scritture concorrenti; accesso al cluster tramite localhost per eseguire workload e iniettare guasti.
- **Non-funzionali:** scenari containerizzati e riproducibili; misurazione e log di tassi di successo; script per inizializzare e ripristinare gli esperimenti.

2 Obiettivi

- Misurare i compromessi tra disponibilità e consistenza durante partizioni se evidenziati.
- Fornire test ripetibili che simulino guasti e producano log leggibili dalla macchina.
- Riportare la correttezza (violazioni di consistenza) e metriche operative.

3 Cosa testeremo: teorema CAP e modelli NoSQL

Il **teorema CAP** afferma che un sistema dati distribuito può fornire al massimo due delle seguenti tre garanzie simultaneamente: Consistenza, Disponibilità e Tolleranza alle partizioni [1]. In pratica: [4]

- **Disponibilità** (availability): ogni richiesta a un nodo non fallito riceve una risposta (senza garantire sia il valore più recente). verrà testato se il sistema continua ad accettare e servire richieste corrette quando alcuni nodi falliscono.
- **Consistenza** (copy-consistency): tutti i nodi vedano gli stessi dati nello stesso momento. verrà testato se le repliche ritornano lo stesso stato dopo diversi eventi casuali su nodi di rete.
- **Tolleranza alle partizioni** (partitioning): il sistema continua a operare nonostante partizioni di rete arbitrarie. verrà testato se il sistema continua a operare nonostante si forzino partizioni di rete arbitrarie.

Gli esperimenti sono eseguiti in un caso ideale; un piccolo cluster containerizzato, i test sono progettati per misurare disponibilità, diverse nozioni di consistenza e comportamento di recupero.

4 Redis data model e implicazioni per i test

Redis è un sistema complesso, noi ne useremo una parte molto semplice, il **String key-value** [3]. Di seguito una panoramica dei tipi di dato principali, coerente con la documentazione ufficiale di Redis [3].

4.1 Tipi principali e proprietà

- **Stringhe:** valori binary-safe per contatori e payload piccoli; comandi atomici 'SET'/'GET'.
- **Hash:** mappe campo/valore per record ad oggetto e aggiornamenti parziali.
- **Liste:** collezioni ordinate (push/pop) per code e log.
- **Set:** collezioni non ordinate di elementi unici con test di appartenenza.
- **Sorted Sets (zsets):** elementi con punteggio per query di range e ranking.
- **Streams:** log append-only con consumer group per processing durevole.

Idealmente il sistema gestisce separatamente le strutture dati dalla logica di partizionamento e replicazione, quindi verrà testato il comportamento di una struttura dati campione.

4.2 Replicazione, persistenza e implicazioni di consistenza

E' stato scelto Redis per la sua semplicità e per la disponibilità di un cluster containerizzato pronto all'uso [3]. Redis supporta replicazione asincrona (primary-replica) [3].

La replicazione asincrona implica che le repliche possano essere in ritardo e restituire dati obsoleti se servite per le letture [3]. Questa tolleranza implica che le letture dalle repliche possano essere stale e quindi la consistenza non è garantita in un caso ideale?

5 Setup sperimentale e metodologia dei test

Gli esperimenti sono eseguiti con orchestration containerizzata (Docker Compose) per garantire riproducibilità.

5.1 Test generico

- **T**: numero di thread concorrenti simulati (thread/processi). Aumentando questo valore si incrementa la concorrenza totale.
- **N**: numero di operazioni locali eseguite da ciascun thread; il totale delle operazioni è $T \times N$.
- **P**: numero di nodi nel cluster Redis; 3 nel nostro caso.

Algorithm 1 Test generico

```
1: procedure TESTGENERICO( $P, T, N$ )
2:   Avviare cluster con  $P$  nodi
3:   Lanciare  $T$  thread worker
4:   for  $t = 1$  to  $T$  do
5:     Ogni worker esegue  $N$  operazioni locali e casuali:
6:     per ciascuna e registra (timestamp, operazione, risultato)
7:   end for
8:   Attendere il completamento dei worker e raccogliere i log
9: end procedure
```

6 Modelli di dati in Redis

Questa sezione fornisce una panoramica sintetica ma completa dei tipi di dato nativi di Redis e delle loro implicazioni per storage, consistenza e comportamento del sistema. L'obiettivo è spiegare come i diversi modelli di dato influenzino la correttezza dell'applicazione e come i test debbano essere progettati per esercitarli.

6.1 Panoramica

Redis è principalmente uno store key-value che espone diversi tipi di dato nativi [3]. Ogni tipo offre garanzie semantiche e pattern d'uso differenti; queste scelte influenzano replicazione, concorrenza e recupero.

6.2 Tipi principali e proprietà

- **Stringhe**: valori binary-safe per contatori e payload piccoli; operazioni atomiche SET/GET.
- **Hash**: mappe campo/valore per record ad oggetto e aggiornamenti parziali.
- **Liste**: collezioni ordinate (push/pop) per code e log temporali.
- **Set**: collezioni non ordinate di elementi unici con test di appartenenza.
- **Sorted set (zset)**: elementi con punteggio per query di intervallo e ranking.
- **Stream**: log append-only con consumer group per elaborazione durevole.
- **Tipi specializzati**: bitmap, HyperLogLog ed estensioni geospaziali per analisi e riepiloghi compatti.

6.3 Data Model example

Questa sezione collega i principali modelli di dato di Redis ai casi d'uso concreti in cui hanno senso, e chiarisce quale problema applicativo ciascuno aiuta a risolvere.

- **Stringhe**: sono la scelta corretta per valori atomici, contatori, cache di piccole dimensioni e flag di stato. Risolvono il problema del salvataggio semplice chiave-valore e supportano operazioni rapide e atomiche come aggiornamenti, letture e incrementi.

- **Hash:** sono utili quando un'entità ha molti attributi, ad esempio un profilo utente, un record prodotto o una configurazione. Risolvono il problema di aggiornare solo alcuni campi senza riscrivere tutto l'oggetto.
- **Liste:** sono adatte a code di lavoro, feed temporali, log append-only e buffer di messaggi. Risolvono il problema dell'ordinamento degli elementi e dell'elaborazione FIFO/LIFO.
- **Set:** sono indicate per rappresentare collezioni di elementi unici, come tag, utenti online o insiemi di appartenenza. Risolvono il problema dei duplicati e rendono efficienti test di membership e operazioni insiemistiche.
- **Sorted set (zset):** sono utili per classifiche, leaderboard, priorità e range di punteggio, ad esempio top utenti o job scheduling. Risolvono il problema dell'ordinamento dinamico con punteggi aggiornabili.
- **Stream:** sono adatti a pipeline di eventi, telemetria e processing distribuito con consumer group. Risolvono il problema dell'ingestione ordinata dei messaggi e della loro elaborazione affidabile.
- **Tipi specializzati:** bitmap, HyperLogLog e strutture geospaziali sono utili per casi come conteggi approssimati, tracciamento di flag compatti e query geografiche. Risolvono problemi di spazio, cardinalità approssimata e rappresentazione efficiente di dati derivati.

Nel nostro esperimento, però, questi modelli non vengono esercitati tutti: il workload di test usa principalmente la struttura **String**, tramite operazioni SET/GET, perché l'obiettivo è misurare disponibilità, consistenza e tolleranza ai guasti del cluster nel caso più semplice e controllabile.

6.4 Replicazione, persistenza e implicazioni di consistenza

Redis supporta la replicazione asincrona (primary-replica). La replicazione asincrona implica che le repliche possano essere in ritardo e possano restituire dati obsoleti se usate per le letture.

In modalità cluster, le chiavi sono partizionate tra i nodi; le operazioni cross-shard richiedono coordinamento accurato oppure aggregazione lato client. Gli stream e i consumer group introducono semantiche di ordinamento e partizionamento che sono rilevanti quando si valuta la consistenza dell'elaborazione dei messaggi.

Conseguenze progettuali:

- Per un comportamento fortemente consistente, i client dovrebbero preferire letture dal primary oppure usare livelli supportati da consenso; in caso contrario, le letture dalle repliche possono essere stale.
- Comandi atomici come HSET e LPUSH riducono alcuni problemi di concorrenza, ma non verranno trattati questi scenari.

7 Setup sperimentale e metodologia dei test

I workload e le procedure automatizzate di test [2], tutti gli esperimenti e procedure verranno eseguite tramite orchestrazione containerizzata (Docker Compose) per garantire riproducibilità. I generatori di carico sono client Python personalizzati; gli script nella cartella `experiments` eseguono letture e scritture secondo gli scenari descritti sotto.

7.1 API e workload

Di seguito sono elencati i componenti principali del sistema di test e le principali chiamate API utilizzate per generare carico e misurare il comportamento del cluster Redis [3]. Non sempre si tratta di chiamate esposte direttamente dal client Redis ufficiale, ma di funzioni wrapper che orchestrano le operazioni e registrano i log.

- **start_stop_container:** script o controlli sui container che arrestano/riavviano i nodi e manipolano le regole di rete, ad esempio con `tc`, per creare partizioni.
- **scan_all:** i client registrano recupera lo stato del nodo, esito e valore osservato per le letture.
- **random_rw_event:** esegue operazioni casuali di lettura e scrittura.
- **random_partition_event:** esegue eventi casuali di partizionamento della rete (spegne/riavvia nodi e legge/scrive dati nel frattempo).
- **random_primary_switch:** esegue eventi casuali di cambio dei nodi primari.

7.1.1 Dettagli dei test

I log grezzi relativi agli esperimenti sono raccolti nella cartella `experiments/logs/` del repository; il riepilogo delle esecuzioni batch è disponibile in `experiments/logs/stats.csv`. Per il collegamento diretto al repository dei log si può usare una path del tipo `https://github.com/USERNAME/REPOSITORY/tree/main/exam/BD/experiments/logs`.

- **Test di consistenza:** le esecuzioni con `random_event` producono un mix di letture, scritture e switch del primario; il comportamento osservato serve a verificare se i tre nodi convergono allo stesso stato oppure se emergono divergenze temporanee durante carico concorrente.
- **Test di disponibilità:** le esecuzioni con `random_ses_event` e `random_primary_switch` simulano stop/start dei container e cambi di nodo attivo; il cluster deve continuare a rispondere alle operazioni di base anche quando un nodo viene momentaneamente tolto dal servizio.
- **Test di partizionamento:** le esecuzioni con `random_partition_event` introducono una partizione controllata e poi la rimuovono; il punto da osservare nei log è se il sistema continua a servire richieste e se, al termine, lo stato dei nodi torna allineato.

8 Discussione

Il file `stats.csv` riassume 36 esecuzioni del piano sperimentale, ottenute variando in modo sistematico il numero di thread concorrenti e il numero di operazioni locali per thread. In tutti i casi il codice di uscita è pari a 0 e l'esito è `ok`, segno che gli scenari provati sono stati eseguiti senza errori operativi e con comportamento stabile lungo l'intera matrice dei test.

Nel complesso, il dataset descrive un sistema che mantiene la propria operatività nelle condizioni osservate, anche al crescere del carico. L'aumento di `thread_count` e di `local_test_count` rende più gravoso l'esperimento, ma non introduce fallimenti visibili nel riepilogo; questo suggerisce una buona robustezza del setup e una coerenza tra orchestrazione, workload e raccolta dei log. Dal punto di vista analitico, il contenuto del CSV è quindi utile soprattutto come evidenza di ripetibilità e come base per confronti quantitativi tra scenari, più che come fonte di anomalie o outlier da interpretare.

9 Conclusioni

Gli esperimenti svolti mostrano che, nello scenario considerato, il cluster Redis esegue in modo stabile le operazioni di base anche in presenza di carico concorrente e di guasti controllati. Il risultato è rilevante per lo studio dei sistemi NoSQL distribuiti perché evidenzia che, in un ambiente containerizzato e ben orchestrato, il sistema osservato riesce a mantenere una buona operatività e a recuperare correttamente dopo gli eventi di disturbo previsti dal piano di test.

Le evidenze raccolte sono in parte coerenti con le aspettative teoriche del modello CAP: durante le condizioni normali il sistema privilegia una gestione efficace delle richieste e, nei casi osservati, non emergono violazioni evidenti nel comportamento complessivo. Tuttavia, questa osservazione non va interpretata come una dimostrazione forte del teorema. L'assenza di errori nel log può dipendere anche dalla natura ideale del test, dalle dimensioni ridotte del cluster, dal tipo di workload scelto e dall'efficienza dello strumento utilizzato per generare e monitorare gli esperimenti, più che da una conferma diretta e generalizzabile delle proprietà CAP.

In altre parole, i risultati indicano soprattutto che il sistema testato si comporta bene nel contesto sperimentale predisposto, ma non bastano da soli a concludere che il comportamento osservato rappresenti in modo completo il compromesso teorico tra consistenza, disponibilità e tolleranza alle partizioni. Per rafforzare questa analisi sarebbe utile estendere il lavoro con cluster più grandi, confrontare Redis con altri sistemi NoSQL con modelli di consistenza differenti e introdurre modelli di guasto più forti, ad esempio partizioni di rete più realistiche, crash simultanei di più nodi, ritardi artificiali nella replica o scenari di recovery più aggressivi.

Tutto questo però esula dallo scopo del progetto.

Riferimenti bibliografici

- [1] Brewer, E. A. (2000). Towards robust distributed systems (cap theorem). Keynote, PODC conference. Available at: <https://people.eecs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf>.
- [2] Jepsen (2026). Distributed systems testing. Accessed 2026-06-19.
- [3] Redis Labs (2024). Redis documentation. Accessed 2026-06-19.
- [4] Wikipedia (2026). Teorema cap — wikipedia, l'enciclopedia libera. [Online; in data 19-giugno-2026].